Phase 1: Preprocessing and Feature Engineering Strategy

Suwayhib: word-level recitation error detection

Faruq Hammed

October 2026 (revised November 2026)

1. Introduction

Suwayhib is an offline Qur'an recitation companion. A learner recites a known verse. The device checks each word and says whether it was recited correctly. It then replays the flagged words next to a reference reciter. This report audits the data behind that per-word decision. It also fixes the preprocessing plan before we write the pipeline. Unless we say otherwise, each number comes from `src/audit.py` (output in `phase1_strategy/audit_output.txt`). A few numbers come from the Phase 2 notebook, and we mark them.

The task is **binary classification**. The target, `is_error_word`, is 1 for a word with an injected error (an **error word**). It is 0 for every other word (a **correct word**).

2. Dataset and Problem Audit

2.1 Dimensions and Origin

The word table has 9,086 rows and 23 columns. That is 22 features (P) and one target.

Quantity	Value
Rows (N), one per word	9,086
Columns	23 = 22 features (P) + 1 target
Recordings / reciters / verses	2,000 / 8 / 458 (surahs 78 to 114)

The data is the Suwayhib test set. Its builder took EveryAyah verse recordings. It found the words with cpfair/quran-align timings. It then put one error at a known word into 1,400 recordings. The other 600 recordings are clean (400) or have a "control seam" that changes nothing (200). Dynamic time warping (DTW) over mel-frequency cepstral coefficients (MFCCs) then aligned each recording to the reference. That gave one row of features per word.

This test set replaces the IqraEval corpus from the proposal. It is already labeled per word, and it matches the scorer that the device runs. IqraEval stays as future work for real learners.

2.2 Variable Inventory

We put every column into exactly one type.

Type	Columns
Continuous (12)	`orig_duration_ms`, `new_duration_ms`, `stretch_rate`, `test_frac`, `ref_frac`, `dtw_cost`, `width`, `dur_ratio`, `dur_z`, `cost_z`, `log_dev`, `suspicion`
Discrete (4)	`word_idx`, `n_words`, `surah`, `ayah`
Binary (1)	`collapsed`
Nominal (4)	`reciter` (8 levels), `error_type` (9), `tier` (4), `split` (2)
Identifier (1)	`recording_id`
Ordinal, datetime, text	none

Each feature has one meaning.

Feature	Definition
`test_frac`, `ref_frac`	The share of the verse duration that the word takes, in the test recording and in the reference
`dur_ratio`	Test word duration divided by reference word duration
`dtw_cost`	Mean DTW alignment cost over the word (acoustic mismatch)
`width`	Number of frames that the DTW path spends on the word
`dur_z`, `cost_z`	`dur_ratio` and `dtw_cost`, standardized within the recording
`log_dev`	|log(`dur_ratio`)|
`suspicion`	The combined per-word score of the Suwayhib scorer
`collapsed`	1 if the DTW path squeezed the word to almost nothing
`orig_duration_ms`, `new_duration_ms`	Length of the reference recording and of the test recording
`stretch_rate`	Time-stretch factor that the generator applied (madd errors only)

`surah` and `ayah` are integers, so by type they are discrete. By role they name the verse, so we treat them as verse identifiers. `word_idx` is a position inside a verse. We count it as discrete and not ordinal, because we do not model it as a rank.

2.3 Baseline Missingness

Only `stretch_rate` has missing values. The other 21 feature columns and the target have none.

Column	Missing count	Missing %
`stretch_rate`	7,106	78.21%
Each of the other 21 feature columns, and the target	0	0.00%

The 7,106 missing values are 3.55% of all 199,892 cells. The table has 1,980 complete rows and 7,106 rows with at least one missing value. `stretch_rate` has a value for every `madd_lengthening` row (889) and every `madd_shortening` row (1,091). No other row has one.

2.4 Target

The table has 7,686 correct words (84.59%) and 1,400 error words (15.41%). The imbalance ratio is 5.49 : 1. The error rate is steady across reciters (13.9% to 17.4%).

We handle the imbalance with `class_weight="balanced"`. We evaluate with PR-AUC and F1. We do not use accuracy, because a model that always says "correct" already scores 84.6%. We reject SMOTE. It mixes words from different recordings and reciters. That makes feature vectors that no recording could produce. If we resample later, we will do it inside the training folds only.

3. Cleaning and Imputation

3.1 Missingness Mechanism

`stretch_rate` is **MNAR**. Its value exists only when the generator added a madd error. So whether it is missing depends on the label we want to predict. It is not MCAR, because `error_type` predicts it perfectly.

We drop `stretch_rate` and do not impute it. An imputed value would invent a stretch that never happened. A missingness flag would equal "has a madd error", and a real device never knows that.

No other feature is missing. We still add a median `SimpleImputer`, fit on the training data, as a safety net for deployment. We use the median because 8 of the 9 numeric features we keep are right-skewed. The mean follows the tail. Mode imputation is not needed, because no categorical input is left. KNN imputation costs too much when nothing is missing.

3.2 Outliers

All 12 features in the table are skewed, and most have heavy tails.

Feature	Skew	Kurtosis	1.5×IQR bounds	IQR outliers	|z| > 3
`dur_z`	19.18	761.5	[−2.70, 2.70]	1,031 (11.3%)	108
`suspicion`	18.28	692.2	[−2.34, 3.89]	938 (10.3%)	118
`cost_z`	3.71	47.6	[−2.70, 2.70]	644 (7.1%)	179
`log_dev`	3.52	19.9	[−0.31, 0.71]	548 (6.0%)	184
`orig_duration_ms`	2.36	7.3	[−1,450, 14,746]	705 (7.8%)	217
`new_duration_ms`	2.30	7.1	[−2,173, 14,832]	595 (6.5%)	212
`n_words`	2.38	6.1	[1, 9]	791 (8.7%)	356
`width`	2.01	10.4	[−68, 276]	329 (3.6%)	112
`dur_ratio`	1.22	9.8	[0.35, 1.65]	556 (6.1%)	102
`test_frac`	0.84	0.4	[−0.17, 0.59]	129 (1.4%)	72
`dtw_cost`	0.80	1.7	[−0.02, 0.65]	129 (1.4%)	71
`ref_frac`	0.63	−0.1	[−0.14, 0.55]	63 (0.7%)	33

Detection. Every feature in the table has |skew| above 0.5. So none is normal, and the z-score rule does not fit. Heavy tails make σ large. This is why the z rule flags only 108 `dur_z` values, while the IQR rule flags 1,031. We use the IQR rule.

Treatment. Some extreme values are real signal here. A deleted or repeated word should give an extreme duration. The 1.5×IQR fence would flatten 10% to 11% of `dur_z` and `suspicion` to one value. So we take two steps:

1. We apply `log1p` to the positive right-skewed features (Section 4.5).
2. We winsorize at Tukey's far-out fences, [Q1 − 3×IQR, Q3 + 3×IQR]. We compute them on the training data only.

The far-out fence clips measurement artifacts, such as `dur_z` = 233.9, which comes from a near-zero reference denominator. It keeps the range where errors live. We cap values and do not delete rows. The extreme rows are mostly error words, and each one still has good values in its other features.

4. Association, Encoding and Scaling

4.1 Feature-Target Tests

We match each test to the type of the feature.

Feature type	Test
Numeric	Point-biserial r (linear), Mann-Whitney U (rank-based, no normality assumption), mutual information (non-linear)
Categorical or binary	χ^2 and Cramér's V

Feature	Point-biserial r	MWU p	MI
dur_ratio	−0.093	1.2e−31	0.056
log_dev	+0.300	3.1e−182	0.056
suspicion	+0.203	5.2e−175	0.051
word_idx	+0.021	2.2e−24	0.048
dur_z	+0.176	7.7e−120	0.048
ref_frac	+0.145	2.7e−41	0.026
dtw_cost	+0.194	3.4e−78	0.017
cost_z	+0.147	2.5e−74	0.016
n_words	−0.127	1.3e−46	0.009
width	+0.017	1.2e−07	0.008
test_frac	+0.070	3.0e−06	0.004
ayah	−0.010	0.35	0.004
new_duration_ms	−0.105	1.0e−25	0.000
orig_duration_ms	−0.102	7.0e−22	0.000
stretch_rate	−0.009	0.62	0.000
surah	−0.005	0.55	0.000

Feature	χ^2	dof	p	Cramér's V
error_type	696.8	8	3.5e−145	0.277
tier	692.1	3	1.1e−149	0.276
reciter	8.5	7	0.29	0.031
split	4.7	1	0.03	0.023
collapsed	0.06	1	0.80	0.003

We rank features by MI, because the strongest signal is not linear. `dur_ratio` has the top MI, but its r is only −0.09. Deletion and truncation make a word shorter. Repetition and lengthening make it longer. So the class means barely move. `log_dev` puts both directions on one side and finds the signal with a linear measure (r = 0.30).

4.2 Leakage Screening

These columns describe how the generator built the data. They do not describe how the recitation sounds.

Column	Decision	Reason
`error_type`, `tier`	Drop (leakage)	They name the error. The error rate is 0.0% for every clean and control recording.
`stretch_rate`	Drop (leakage)	It has a value only for madd errors.
`word_idx`	Drop (artifact)	The first word is never an error word (0 of 2,000 first words). Its MI of 0.048 comes from the generator's rule. Real learners do make first-word errors.
`split`	Drop	Generator bookkeeping.
`collapsed`	Drop	Near-zero variance: 14 of 9,086 rows are 1 (χ^2 p = 0.80).
`recording_id`, `reciter`	Grouping keys only	`reciter` has no link to the target (p = 0.29). No learner is one of these 8 reciters.
`surah`, `ayah`	Not features	They name the verse and have no link to the target (p = 0.55 and 0.35).

4.3 Multicollinearity

We flag pairs with |Pearson r| or |Spearman ρ| of at least 0.85. We also flag features with VIF above 10. Spearman catches monotonic overlap that skew hides from Pearson. From each flagged pair we keep the feature with the higher MI. Then we compute VIF again.

Pair	Pearson / Spearman	VIF	Decision
`orig_duration_ms`, `new_duration_ms`	+0.935 / +0.876	12.6 / 12.4	Replace both with `rec_len_ratio` = new ÷ orig, since each alone has MI near 0
`test_frac`, `ref_frac`	+0.852 / +0.866	15.4 / 12.1	Drop `test_frac` (MI 0.004 against 0.026), because `dur_ratio` already compares it to the reference
`dur_z`, `suspicion`	+0.917 / +0.642	13.7 / 15.0	Drop `dur_z` (MI 0.048 against 0.051), because `suspicion` already contains it

After these drops every VIF is at most 2.55. Nine inputs are left: `ref_frac`, `dtw_cost`, `width`, `dur_ratio`, `cost_z`, `log_dev`, `suspicion`, `n_words`, and `rec_len_ratio`. The VIF after the drops comes from the Phase 2 notebook.

4.4 Encoding

No categorical column is left as an input, so we need no encoder. If we had kept `reciter` (8 unordered levels), we would have used one-hot encoding. The data has no ordinal variables. Even `tier` has no natural order, so ordinal encoding does not apply.

4.5 Transformation and Scaling

We apply `log1p` to `width`, `dur_ratio`, `suspicion`, and `n_words`. These features are positive and right-skewed. For example, the skew of `suspicion` drops from 18.28 to 1.49. We do not apply the log to `rec_len_ratio`. It is left-skewed (−0.76), and `log1p` would make the skew worse (−1.35).

We then apply StandardScaler. The L2 penalty of logistic regression and PCA both need features on one scale. We reject MinMaxScaler. One extreme value squeezes the rest of a feature into a narrow band, and nothing in the plan needs a [0, 1] range. Trees do not care about scale, and we plan to compare them.

5. Split and Leakage Prevention

5.1 Split

A random row split would leak. Words from one recording share a voice, a noise profile, and the within-recording scaling of `cost_z` and `dur_z`. The device also faces another question. Does the model work on a voice it has never heard? A split by reciter handles both problems, because each recording has one reciter.

Partition	Reciters	Rows	Share	Error rate
Train	6	6,763	74.4%	14.9%
Test	2 (Husary, Minshawy)	2,323	25.6%	16.8%

The test-set metadata (`testset_meta.json`) lists the same two reciters as its own holdout. The audit statistics in this report use all rows, so they include these two reciters. Phase 2 (Section V) repeats the audit on the six training reciters only (`python src/audit.py --train-only`). No drop changes.

Two test reciters keep the test score from reflecting one voice. The test set has 391 error words, so the F1 estimate is stable. The sample is too small for a separate validation set.

We do not force stratification. Grouping comes first, and it already keeps the error rates close (14.9% and 16.8%). The data has no timestamps and no recording order, so a time-based split does not apply.

5.2 Validation

We use leave-one-reciter-out cross-validation (`LeaveOneGroupOut`, 6 folds). We pick hyperparameters by mean PR-AUC. We use the test reciters once, at the end.

5.3 Leakage-Prevention Architecture

```
Pipeline([
  ("prep", ColumnTransformer([
      ("num", Pipeline([
          ("impute",  SimpleImputer(strategy="median")),    # fit on train
          ("log",     Log1pTransformer(columns=LOG_COLS)),  # stateless
          ("cap",     IQRWinsorizer(k=3.0)),                # fit on train
          ("scale",   StandardScaler()),                    # fit on train
      ]), NUMERIC_FEATURES),
  ], remainder="drop")),
  ("model", LogisticRegression(class_weight="balanced")),
])
```

The design follows four rules:

1. We split the raw table before any fitting.
2. We fit on `X_train` only. `X_test` goes through `transform` and `predict` only.
3. We keep the column drops as a fixed list. We do not learn them from data.
4. We cross-validate the full pipeline. Each fold refits its own medians, fences, and scaler.

`cost_z` and `dur_z` are scaled within each recording, and no recording is in both partitions. So they do not leak.

6. Limitation

Every error is a synthetic splice into the audio of a professional reciter. Real learners, especially Hausa-accented or Fulfulde-accented children, will make errors that this data does not have. So the Phase 2 results are an upper bound until we test on IqraEval and on field recordings.